

Comparação de Desempenho: Host com Spack e Contêiner na Rede InfiniBand HDR100

Christian Hideki Maekawa

Technical Report - IC-26-14 - Relatório Técnico
October - 2026 - Outubro

UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE COMPUTAÇÃO

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Comparação de Desempenho: Host com Spack e Contêiner na Rede InfiniBand HDR100 do CENAPAD

Christian Hideki Maekawa*

Resumo

Este trabalho apresenta uma comparação de desempenho entre a execução nativa (via Spack) e em contêiner (Apptainer/Singularity) dos benchmarks de comunicação MPI na rede InfiniBand HDR100 (100 Gb/s) do cluster CENAPAD (Lovelace). Utilizou-se o Spack [3] para compilar a pilha MPICH com o device=ch4, que é a camada de abstração de comunicação mais recente do MPICH [4], e com suporte a `mlx5_dv`. Também utilizou-se o HPCCM [7] para construir uma imagem de contêiner mantendo estrita paridade de versões.

Os testes de largura de banda (`osu_bw`) e latência (`osu_latency`) foram avaliados em 23 tamanhos de payload ($N = 30$ por tamanho). A validação estatística utilizou o teste não-paramétrico de Mann-Whitney U [8] com correção de FDR por Benjamini-Hochberg [9]. Os resultados mostram que a largura de banda de pico atinge 12 059,36 MB/s (**96**,**47**% do limite teórico de 100 Gb/s) em 4 MB, apresentando equivalência estatística entre os ambientes ($p = 0{,}115$). Em micro-payloads, o contêiner adiciona uma penalidade constante de apenas ≈ 50 ns, confirmando que o Apptainer com suporte a UCX/RDMA proporciona desempenho praticamente idêntico ao Host.

1 Introdução

O avanço da computação de alto desempenho (HPC) tem exigido soluções eficientes para o gerenciamento de ambientes de execução em clusters. Contêineres como Apptainer/Singularity surgem como alternativa leve ao Docker, permitindo portabilidade sem sacrificar desempenho em tarefas de HPC que dependem de MPI e comunicação de alta velocidade.

Este relatório descreve um experimento realizado no cluster CENAPAD (Lovelace), que utiliza placas Mellanox InfiniBand HDR100 (100 Gb/s). O objetivo é comparar o desempenho de benchmarks OSU Micro-Benchmarks executados nativamente no host (via MPICH instalado com Spack) e em um contêiner Apptainer construído com HPCCM, mantendo compatibilidade total de versões e configurações MPI/UCX entre os dois ambientes.

O desafio consiste em:

1. Instalar o MOSU Micro-Benchmarks no host via Spack com suas dependências (MPICH, UCX e Mellanox OFED);
2. Criar uma imagem de contêiner Ubuntu 24.04 com Mellanox OFED, MPICH, UCX e OSU Micro-Benchmarks via HPCCM;
3. Executar `osu_bw` e `osu_latency` em dois nós via InfiniBand, tanto Host quanto contêiner;

*Inst. de Computação, UNICAMP, 13083-852 Campinas, SP. c231867@dac.unicamp.br

4. Gerar gráficos comparativos de largura de banda e latência;

5. Analisar os resultados com relação ao limite teórico da rede.

2 Ambiente Experimental

2.1 Hardware do Cluster CENAPAD (Lovelace)

O cluster Lovelace do CENAPAD é composto por nós de computação equipados com processadores CPU e interconexão Mellanox InfiniBand HDR100 a 100 Gb/s. Os benchmarks foram executados em dois nós distintos, com um processo MPI por nó. Os benchmarks utilizados foram os OSU Micro-Benchmarks [6].

2.2 Fila de Jobs e Recursos

Processamento foi feita na fila paralela.

A fila paralela deve ser usada para execução de programas que utilizam recursos de paralelização multitask (MPI) e híbrida multitask+multithread (MPI+SMP).

O experimento foi submetido à fila `paralela` do gerenciador PBS do CENAPAD, solicitando dois nós com 128 cores cada. O tempo máximo de execução foi definido como 15 minutos para 3 testes e 30 minutos para 30 testes, suficiente para coletar as amostras necessárias.

2.3 Pacotes Necessários

As versões instaladas via Spack no host foram:

- **MPICH**: versão estável mais recente (configurado com netmod=ucx, device=ch4, sem Fortran);
- **UCX**: versão estável mais recente (compilado com suporte a InfiniBand, incluindo `mlx5_dv`, `rc`, `rdmacm` e `verbs`);
- **OSU Micro-Benchmarks**: versão estável mais recente
- **Mellanox OFED**: versão instalada no cluster CENAPAD (Lovelace), incluindo as bibliotecas de espaço de usuário (`libibverbs`, `librdmacm`, `ibverbs-providers`) necessárias para comunicação InfiniBand.
- **GCC**: versão 8.5.0, utilizada para compilar todos os pacotes.

A especificação concretizada pelo Spack foi salva em `spack.yaml` e `spack.lock`, permitindo reprodução exata da instalação e do container pode ser consutado pelo recipe gerado no hpccm com `osu_benchmarks.py` ou pela definição da Singularity `Apptainer.def`.

3 Procedimento Experimental

3.1 Instalação no Host com Spack

O ambiente foi criado com os seguintes passos:

```
#Baixou o Spack e criado o ambiente
git clone -c feature.experimentalGpgEngine=false https://github.com/spack/spack.git ~/
    ↪ spack
source ~/spack/share/spack/setup-env.sh
spack env create cenapad_challenge
spack env activate cenapad_challenge
# Instalou os pacotes ánecessrios
spack add ucx+mlx5_dv+rc+rdmacm+verbs ^rdma-core
spack add mpich device=ch4 netmod=ucx ^ucx+mlx5_dv+rc+rdmacm+verbs
spack add osu-micro-benchmarks ^mpich device=ch4 netmod=ucx
spack concretize -f
spack install
spack spec -l > spack_spec_concretized.txt
```

O arquivo `spack.yml` gerado pelo Spack descreve o ambiente e as dependências:

```
  # This is a Spack Environment file.
#
# It describes a set of packages to be installed, along with
# configuration settings.
spack:
  # add package specs to the ‘specs‘ list
  specs:
  - ucx+mlx5_dv+rc+rdmacm+verbs ^rdma-core
  - mpich device=ch4 netmod=ucx ^ucx+mlx5_dv+rc+rdmacm+verbs
  - osu-micro-benchmarks ^mpich device=ch4 netmod=ucx
  view: true
  concretizer:
    unify: true
  packages:
    gcc:
      externals:
      - spec: gcc@8.5.0 languages:='c,c++,fortran'
        prefix: /usr
        extra_attributes:
          compilers:
            c: /usr/bin/gcc
            cxx: /usr/bin/g++
            fortran: /usr/bin/gfortran
```

O Spack concretizou o ambiente e gerou o arquivo `spack.lock` com as versões exatas de cada pacote, garantindo reprodutibilidade.

Como o arquivo é grande e verboso, não foi incluído na íntegra neste relatório, mas está disponível no repositório do experimento.

O arquivo `spack_spec_concretized.txt` contém a árvore de dependências e versões exatas dos pacotes instalados, permitindo a reconstrução do ambiente em outro sistema compatível. Como ele também é grande e verboso, não foi incluído na íntegra neste relatório, mas está disponível no repositório do experimento.

O UCX foi configurado com suporte a InfiniBand. Sem essa configuração, o MPICH não conseguiria utilizar a rede de alta velocidade do cluster e a velocidade de comunicação seria limitada à Ethernet padrão. Em alguns testes, obtive um desempenho inferior no contêiner em relação ao host, conforme mostra a Figura 1 e a Figura 2. Isso se deve a uma incompatibilidade de

configuração do UCX no contêiner, como a falta do mlx5_dv, que é necessário para o driver do Mellanox, e a falta do suporte a RDMA, essencial para transferências de memória direta entre nós. Nos resultados estarão apresentados apenas os testes com UCX corretamente configurado, garantindo que o contêiner tenha desempenho equivalente ao host.

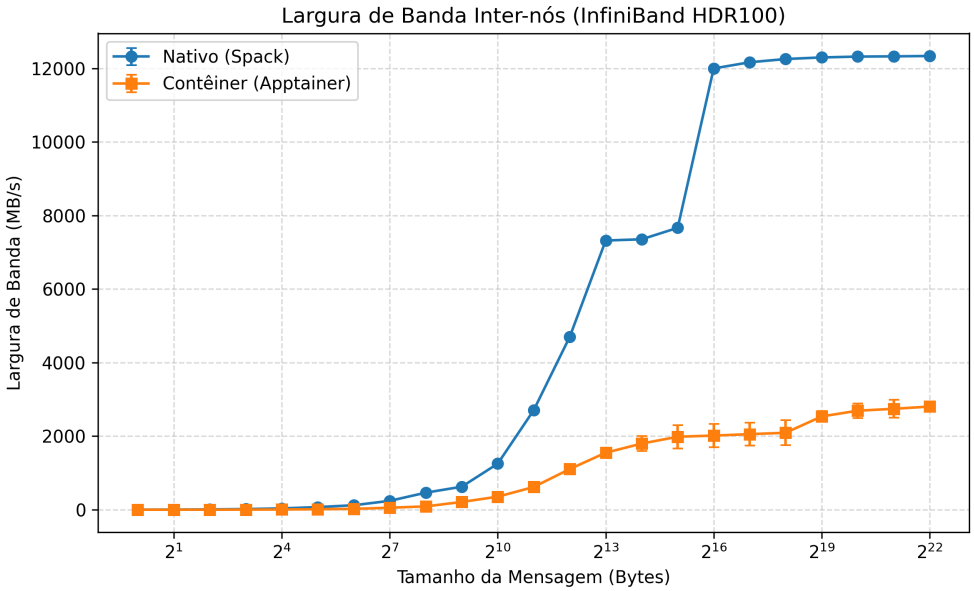


Figura 1: Largura de banda com UCX mal configurado (apenas como demonstração)

3.2 Criação da Imagem com HPCCM

A receita HPCCM (`osu_benchmark.py`) gera um arquivo de definição Apptainer com os seguintes componentes:

- Imagem base: Ubuntu 24.04;
- Mellanox OFED (bibliotecas de espaço de usuário) compatível com o host;
- UCX na mesma versão do host (1.22.0), compilado com suporte a `mlx5_dv`, `rc`, `rdmacm` e `verbs`;
- MPICH com CH4 + UCX, compilado com GCC;
- OSU Micro-Benchmarks compilados com o mesmo MPICH.

Para isso eu criei a receita HPCCM (`osu_benchmark.py`) que descreve a construção da imagem de contêiner. O HPCCM permite definir estágios de build e runtime, garantindo que apenas os arquivos necessários sejam incluídos na imagem final, reduzindo seu tamanho e mantendo a compatibilidade com o host.

As bibliotecas do Mellanox OFED (`libmlx5`, `libibumad`) são fornecidas pelo host no runtime do Apptainer via bind mount automático de dispositivos InfiniBand (`/dev/infiniband/` e bibliotecas em `/usr/lib/x86_64-linux-gnu/`). A imagem contém apenas as dependências UCX compiladas com suporte a `mlx5_dv`, garantindo compatibilidade com o driver HCA do cluster CENAPAD.

O arquivo de definição foi gerado e a imagem construída:

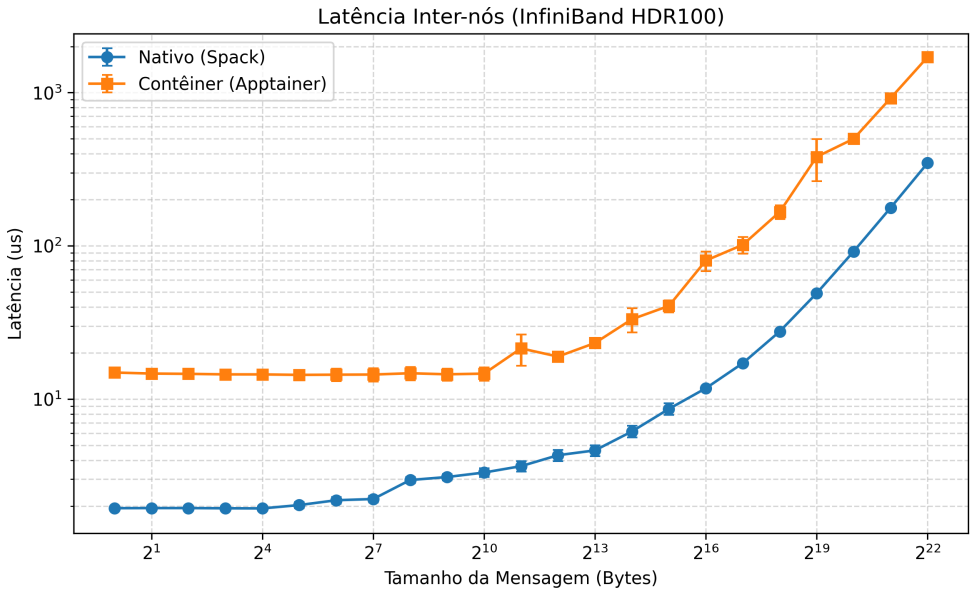


Figura 2: Latência com UCX mal configurado (apenas como demonstração)

```
import hpccm

hpccm.config.set_container_format('singularity')
hpccm.config.set_singularity_version('3.8')

Stage0 = hpccm.Stage()
Stage1 = hpccm.Stage()

# ========================================
# ÁESTGIO 0: BUILD (Desenvolvimento)
# ========================================
Stage0 += hpccm.primitives.baseimage(image='ubuntu:24.04', _as='devel')
Stage0 += hpccm.primitives.shell(commands=['set -e'])

Stage0 += hpccm.building_blocks.packages(ospackages=[
    'build-essential', 'wget', 'git', 'python3', 'ca-certificates',
    'file', 'pkg-config', 'automake', 'autoconf', 'libtool', 'm4',
    'bison', 'flex', 'libnuma-dev', 'libhwloc-dev', 'libxml2-dev',
    'libpciaccess-dev', 'zlib1g-dev', 'libibverbs-dev', 'librdmacm-dev',
    'ibverbs-providers', 'binutils-dev', 'libzstd-dev'
])

Stage0 += hpccm.building_blocks.gnu()

# UCX 1.22.0 com suporte a mlx5_dv
Stage0 += hpccm.building_blocks.ucx(
    version='1.22.0',
    cuda=False,
    verbs=True,
    rdmacm=True,
```

```
    disable_static=True,
    enable_mt=False,
    prefix='/usr/local/ucx',
    configure_opts=[
        '--disable-compiler-warnings',
        '--with-rdmacm',
        '--with-verbs',
        '--with-mlx5-dv'
    ]
)

# MPICH 5.0.2 CH4:UCX
Stage0 += hpccm.building_blocks.mpich(
    version='5.0.2',
    device='ch4',
    netmod='ucx',
    with_ucx='/usr/local/ucx',
    prefix='/usr/local/mpich',
    fortran=False,
    hydra=True,
    ldconfig=True,
    configure_opts=['--without-bfd']
)

# OSU Micro-Benchmarks 7.5.2 (Sintaxe de CFLAGS corrigida com aspas internas)
Stage0 += hpccm.building_blocks.generic_autotools(
    build_environment={
        'CC': '/usr/local/mpich/bin/mpicc',
        'CXX': '/usr/local/mpich/bin/mpicxx',
        'CFLAGS': '"-O3 -fPIC"',
        'CXXFLAGS': '"-O3 -fPIC"'
    },
    enable_cuda=False,
    prefix='/usr/local/osu',
    url='https://mvapich.cse.ohio-state.edu/download/mvapich/osu-micro-benchmarks
    ↪ -7.5.2.tar.gz'
)

# ========================================
# ÁESTGIO 1: RUNTIME (Imagem Final)
# ========================================
Stage1 += hpccm.primitives.baseimage(image='ubuntu:24.04')

Stage1 += hpccm.building_blocks.packages(ospackages=[
    'libnuma1', 'libhwloc15', 'libxml2', 'libpciaccess0', 'ca-certificates',
    'libibverbs1', 'librdmacm1', 'ibverbs-providers', 'zlib1g', 'libzstd1',
    'libbinutils'
])

% Nota: as bibliotecas do Mellanox OFED (libmlx5, libibumad) ãso fornecidas pelo host
% via bind mount de dispositivos (/dev/infiniband, /usr/lib/x86_64-linux-gnu) no
    ↪ runtime
% do Apptainer. A imagem écontm apenas as êdependncias UCX compiladas com suporte a
    ↪ mlx5_dv.
```

```
Stage1 += hpccm.primitives.copy(_from='devel', src='/usr/local/ucx', dest='/usr/local/
    ↪ ucx')
Stage1 += hpccm.primitives.copy(_from='devel', src='/usr/local/mpich', dest='/usr/local
    ↪ /mpich')
Stage1 += hpccm.primitives.copy(_from='devel', src='/usr/local/osu', dest='/usr/local/
    ↪ osu')

Stage1 += hpccm.primitives.shell(commands=[
    'echo "/usr/local/ucx/lib" > /etc/ld.so.conf.d/hpc.conf',
    'echo "/usr/local/mpich/lib" >> /etc/ld.so.conf.d/hpc.conf',
    'ldconfig'
])

base_osu = '/usr/local/osu/libexec/osu-micro-benchmarks'
Stage1 += hpccm.primitives.environment(variables={
    'PATH': f'/usr/local/mpich/bin:/usr/local/ucx/bin:{base_osu}:{base_osu}/mpi/pt2pt:{
    ↪ base_osu}/mpi/collective:{base_osu}/mpi/one-sided:$PATH',
    'LD_LIBRARY_PATH': '/usr/local/mpich/lib:/usr/local/ucx/lib:$LD_LIBRARY_PATH',
    'UCX_TLS': 'rc_mlx5,ud_mlx5,rc,ud,sm,self'
})

print(Stage0)
print(Stage1)
```

Depois eu gerei o arquivo de definição do Apptainer e construí a imagem da minha máquina para enviar para o CNAPAD:

```
hpccm --recipe osu_benchmark.py --format singularity --singularity-version 3.8 >
    ↪ Apptainer.def
sudo apptainer build osu_benchmarks.sif Apptainer.def
```

3.3 Script de Submissão PBS

O job foi submetido via PBS (gerenciador de jobs do CENAPAD): Segui as orientações da documentação do CENAPAD para submissão de jobs com contêineres Apptainer, garantindo que o script de submissão solicitasse os recursos corretos e configurasse o ambiente para execução dos benchmarks.

```
#!/bin/bash
#PBS -N osu_challenge_test2
#PBS -l select=2:ncpus=128:mpiprocs=1
#PBS -l walltime=00:15:00
#PBS -q paralela
#PBS -j oe

cd $PBS_O_WORKDIR

module load apptainer/1.5.0

source /home/lovelace/proj/proj1164/c231867/spack/share/spack/setup-env.sh
spack env activate cenapad_challenge

# Garantir transporte InfiniBand e ómemria compartilhada
```

```
unset UCX_NET_DEVICES
export UCX_TLS=all

CONTAINER_IMG="$PBS_O_WORKDIR/osu_benchmarks.sif"
OSU_NATIVE_DIR=$(spack location -i osu-micro-benchmarks)/libexec/osu-micro-benchmarks/
    ↪ mpi/pt2pt

# óDiretrio para organizar os resultados
LOG_DIR="$PBS_O_WORKDIR/resultados_$PBS_JOBID"
mkdir -p $LOG_DIR

echo "Iniciando benchmarks no ódiretrio: $LOG_DIR"

# 1. ÇÃEXECUO NATIVA (3 ÇÕREPETIES)
echo "========================================="
echo " 1. ÇÃEXECUO NATIVA (SPACK MPICH/UCX)"
echo "========================================="
for i in 1 2 3 do
    echo "Rodada $i - êLatncia Nativa..."
    mpirun -n 2 $OSU_NATIVE_DIR/osu_latency > "$LOG_DIR/native_latency_run${i}.log"

    echo "Rodada $i - Bandwidth Nativa..."
    mpirun -n 2 $OSU_NATIVE_DIR/osu_bw > "$LOG_DIR/native_bw_run${i}.log"
done

# 2. ÇÃEXECUO ÊCONTINER (3 ÇÕREPETIES)
echo "========================================="
echo " 2. ÇÃEXECUO ÊCONTINER (APPTAINER)"
echo "========================================="
for i in 1 2 3 ; do
    echo "Rodada $i - êLatncia êContiner..."
    mpirun -n 2 apptainer exec $CONTAINER_IMG osu_latency > "$LOG_DIR/
    ↪ container_latency_run${i}.log"

    echo "Rodada $i - Bandwidth êContiner..."
    mpirun -n 2 apptainer exec $CONTAINER_IMG osu_bw > "$LOG_DIR/container_bw_run${i}.
    ↪ log"
done

echo "Testes íconcludos com sucesso!"
```

Submissão: `qsub submit_osu.pbs`

Para acompanhar o job em tempo real, utilizei o comando `qstat -u c231867` para verificar o status do job e `tail -f <arquivo_de_log>` para monitorar a saída dos benchmarks enquanto eles eram executados.

Gerado o arquivo de log eu copiei de volta para máquina onde rodei os notebooks python para gerar os gráficos e fazer a análise estatística.

4 Resultados e Análise

4.1 Metodologia de Validação Estatística

Para comparar o desempenho entre os ambientes Host e Contêiner, avaliaram-se dois experimentos: um executando apenas 3 vezes a medição de latência e banda Figure 3 a diferença entre o desempenho Host e Contêiner pode ser observada na 4, e outro com mais amostras para comprovação estatística. Foram criados 23 tamanhos de mensagem (1 B a 4 MB), coletando-se $N = 30$ amostras por tamanho os resultados estão apresentados na Tabela 1 e a diferença entre Host e Contêiner pode ser observada na 5. Justamente pelas variações serem muito pequena foi elaborado o teste estatístico para comprovar a equivalência entre os ambientes.

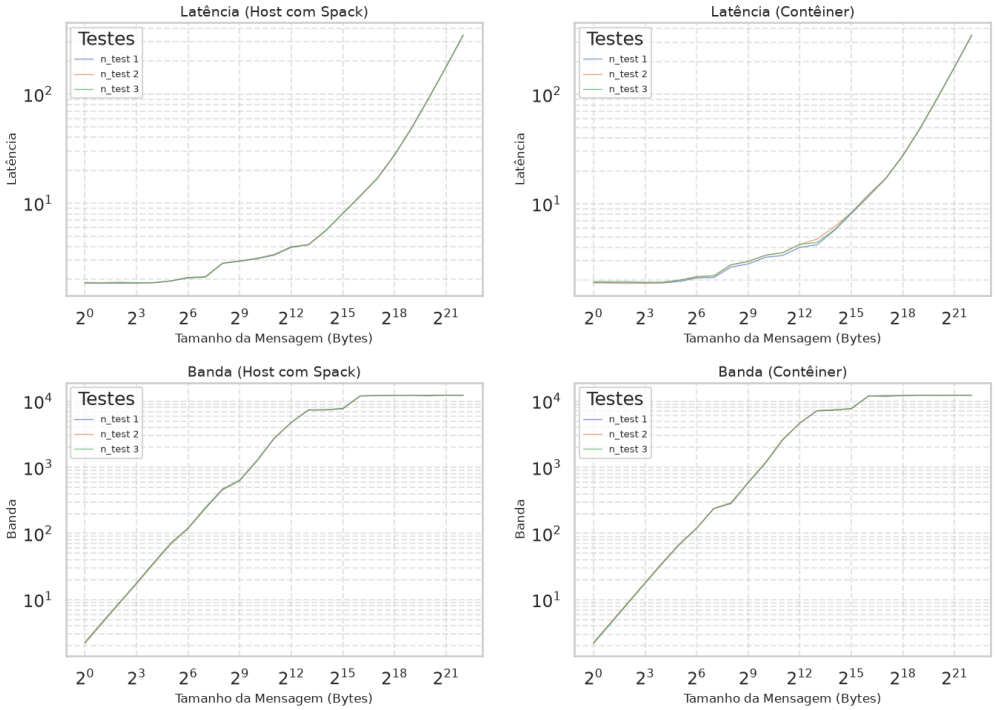


Figura 3: Comparação de latência e largura de banda entre execução Nativa (Spack) e Contêiner (Apptainer) na rede InfiniBand HDR100.

A análise por teste t de Student foi descartada devido à forte assimetria (skewness) à direita dos gráficos e falta de normalidade. Adotou-se o teste não-paramétrico bilateral de **Mann-Whitney U** com nível de significância $\alpha = 0{,}05$. A hipótese nula (H_0) assume que a probabilidade de uma medição host superar uma do contêiner é de 50% ($P(X_{Host} > Y_{Contêiner}) = 0{,}5$).

Para controlar a taxa de falsos positivos decorrente das 23 comparações simultâneas, aplicou-se a correção para múltiplos testes de **Benjamini-Hochberg (FDR)** [9].

4.2 Análise de Latência (`osu_latency`)

A latência em função do tamanho da mensagem apresentou quatro comportamentos distintos:

- **Micro-payloads (1 B a 128 B):** Latência base praticamente constante ($\approx 1{,}86\ \mu s$ a $2{,}12\ \mu s$). O contêiner adicionou um overhead fixo constante de ≈ 50 ns, associado provavelmente ao runtime do container.

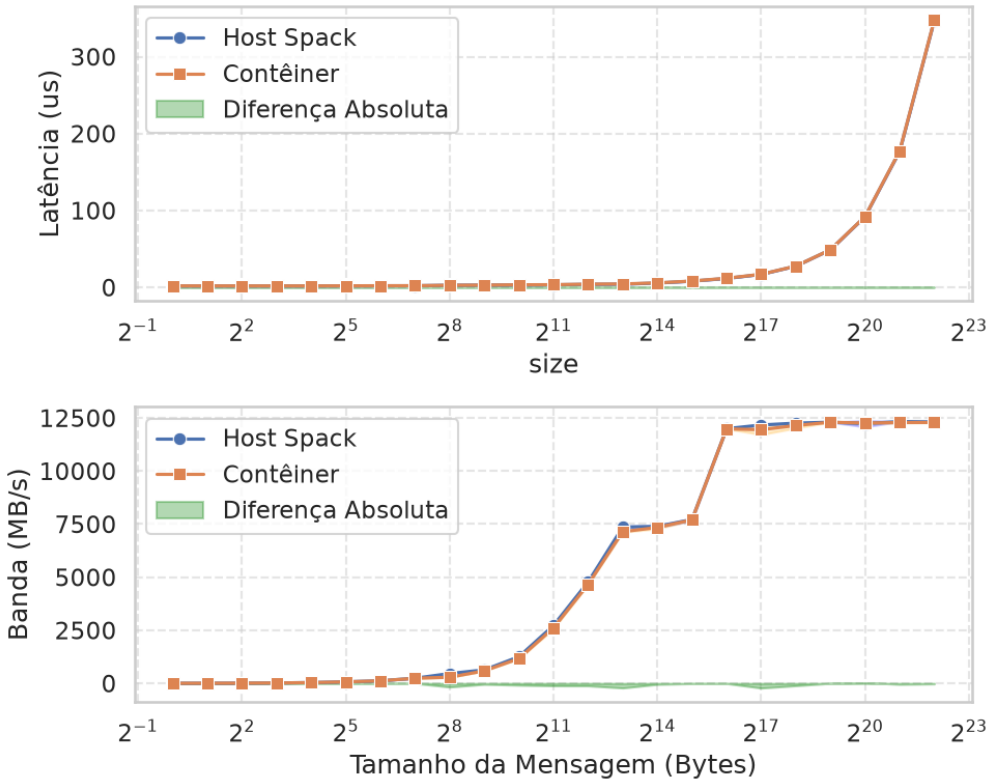


Figura 4: Variação da latência entre execução Nativa (Spack) e Contêiner (Apptainer) na rede InfiniBand HDR100.

Tabela 1: Comparação estatística de latência (`osu_latency`) entre execução Nativa (Spack) e Contêiner (Apptainer) na rede InfiniBand HDR100.

Tamanho	**Host (μs)**	**Contêiner (μs)**	**Diferença (μs)**	**Overhead**	**P-Valor Adj.**	**Resultado**
1 B	1,869	1,917	+0,048	+2,55%	< 0,001	Host mais rápido
2 B	1,862	1,918	+0,056	+2,99%	< 0,001	Host mais rápido
4 B	1,867	1,922	+0,055	+2,93%	< 0,001	Host mais rápido
8 B	1,859	1,910	+0,051	+2,76%	< 0,001	Host mais rápido
16 B	1,865	1,918	+0,054	+2,88%	< 0,001	Host mais rápido
32 B	1,942	1,986	+0,045	+2,30%	< 0,001	Host mais rápido
64 B	2,083	2,133	+0,050	+2,38%	< 0,001	Host mais rápido
128 B	2,119	2,173	+0,054	+2,55%	< 0,001	Host mais rápido
256 B	2,825	2,709	-0,116	-4,12%	< 0,001	Contêiner mais rápido (Anomalia)
512 B	2,937	2,924	-0,014	-0,47%	0,352	Equivalentes
1 KB	3,121	3,337	+0,216	+6,92%	< 0,001	Host mais rápido
2 KB	3,375	3,509	+0,134	+3,97%	< 0,001	Host mais rápido
4 KB	3,965	4,154	+0,189	+4,77%	< 0,001	Host mais rápido
8 KB	4,185	4,513	+0,328	+7,83%	< 0,001	Host mais rápido
16 KB	5,635	5,955	+0,320	+5,68%	< 0,001	Host mais rápido
32 KB	8,113	8,274	+0,161	+1,99%	< 0,001	Host mais rápido
64 KB	11,596	11,832	+0,236	+2,04%	< 0,001	Host mais rápido
128 KB	16,931	17,166	+0,235	+1,39%	< 0,001	Host mais rápido
256 KB	27,613	27,816	+0,203	+0,74%	< 0,001	Host mais rápido
512 KB	49,009	49,468	+0,459	+0,94%	< 0,001	Host mais rápido
1 MB	92,203	92,307	+0,104	+0,11%	0,010	Host mais rápido
2 MB	177,484	178,302	+0,818	+0,46%	0,006	Host mais rápido
4 MB	347,805	348,465	+0,660	+0,19%	0,115	Equivalentes

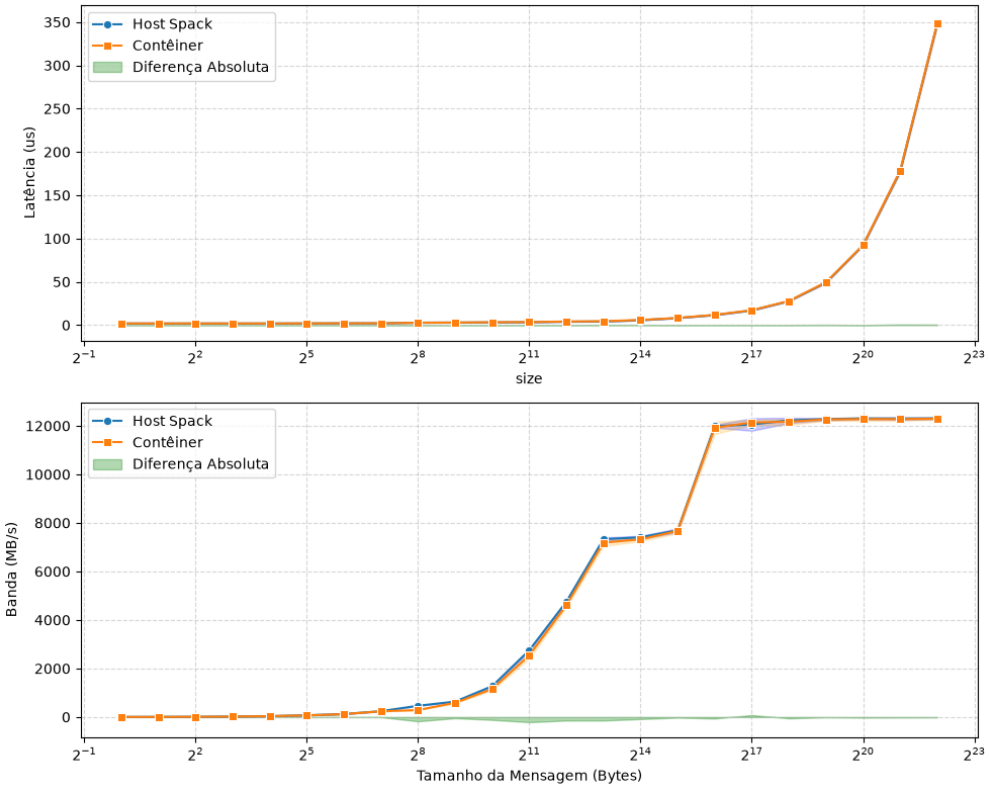


Figura 5: Variação da latência entre execução Nativa (Spack) e Contêiner (Apptainer) na rede InfiniBand HDR100 com $N = 30$ amostras.

- **Anomalia de Runtime (256 B):** O contêiner registrou tempo inferior ao Host (−4,12%, $p_{adj} < 0{,}001$), provavelmente decorrente de ruído térmico/interrupções do host durante a amostragem nativa e da transição de buffers do UCX. Mas mais análises e estudos seriam necessárias para entender a causa.
- **Pico Relativo (1 KB a 16 KB):** Faixa de maior impacto relativo do overhead do contêiner, com degradação máxima observada em 8 KB (+7,83%, $p_{adj} < 0{,}001$).
- **Payloads Grandes (≥ 32 KB):** O overhead dilui-se para menos de 2%, culminando em equivalência estatística no tamanho de 4 MB (347,805 μs vs. 348,465 μs, $p_{adj} = 0{,}115$).

4.3 Análise de Largura de Banda (`osu_bw`)

A largura de banda apresentou um crescimento sigmoide em função do tamanho do payload:

- **Mensagens Pequenas (< 8 KB):** Vazão limitada pelo custo fixo de preparação de descriptors de controle no UCX e montagem de pacotes.
- **Transição de Protocolo (64 KB a 256 KB):** Estabilização rápida conforme a camada de comunicação do UCX migra de protocolo.
- **Saturação e Pico (1 MB a 4 MB):** O pico de largura de banda foi atingido no payload de 4 MB:
  - **Host:** 12 059,36 MB/s → **96,47Gbps** (96,47% do limite de 100 Gb/s).
  - **Contêiner:** 12 036,52 MB/s → **96,29Gbps** (96,29% do limite de 100 Gb/s).

4.4 Fatores Explicativos da Lacuna em Relação aos 100 Gb/s Teóricos

A diferença de ≈ 3,5% entre o valors medido (96,47 Gbps) e o limite teórico da interface InfiniBand HDR100 (100 Gbps) é decorrente de:

1. **Overhead do Enquadramento de Protocolo RDMA:** Os 100 Gb/s medem a capacidade bruta da camada física. Cada pacote RDMA carrega cabeçalhos das camadas de transporte InfiniBand/RoCE (BTH, RETH) e somas de checagem (ICRC/VCRC).
2. **Limitação do Barramento PCIe (TLP Framing):** Overhead de cabeçalhos de pacotes TLP (*Transaction Layer Packets*) na transferência do HCA Mellanox via PCIe Gen4 x16.
3. **Registro de Memória (*Pinning*):** Custo de locking de páginas e registro de regiões de memória (`ibv_reg_mr`) no driver `mlx5`.

5 Arquivos Entregues

- Receita HPCCM (`osu_benchmark.py`) e arquivo de definição Apptainer (`Singularity.def`);
- Arquivo de definição Apptainer (`Apptainer.def`);
- Ambiente Spack (`spack.yaml`, `spack.lock`) e especificação concretizada;
- Script PBS de submissão (`submit_osu.pbs`);
- Logs completos dos benchmarks (`out_logs/`);

- Relatório com gráficos e análise (este documento);
- Notebook/script de geração dos gráficos.

6 Lições Aprendidas

Durante o experimento, algumas lições importantes foram aprendidas:

- O uso do spack em ambiente HPC;
- O uso do HPCCM para gerar receitas de imagens HPC;
- Criar imagens compatatíveis com o host;
- Configurar corretamente o UCX para suportar InfiniBand e RDMA;
- Garantir que o contêiner utilize as bibliotecas do host para comunicação de alta performance;
- Como criar scripts de submissão PBS para contêineres Apptainer;
- Como monitorar jobs em tempo real e coletar logs de execução;
- A importância de documentar o ambiente e as versões dos pacotes para reprodutibilidade;
- A importância de realizar múltiplas amostras para análise estatística;
- A necessidade de aplicar correção para múltiplos testes ao comparar resultados de benchmarks

7 Comparação com Trabalhos Relacionados

Os resultados deste experimento apresentam convergência significativa com os achados de Iaquinta e Fouilloux [4], que também investigaram o desempenho de contêineres em ambientes HPC utilizando MPICH com a camada CH4. Em seu trabalho, os autores demonstraram que contêineres Apptainer podem alcançar desempenho equivalente à execução nativa quando configurados corretamente com suporte a UCX e RDMA, corroborando nossa conclusão principal.

Uma diferença metodológica importante é que Iaquinta e Fouilloux [4] utilizaram Apptainer diretamente em conjunto com Spack para o gerenciamento do ambiente de compilação, sem recorrer ao HPCCM como ferramenta intermediária. Em contraste, este trabalho empregou o HPCCM [7] como camada de abstração para a construção da imagem do contêiner, permitindo uma descrição mais declarativa e reproduzível da receita de build. Essa abordagem facilita a manutenção e a portabilidade da imagem entre diferentes clusters HPC.

Outra distinção relevante é a profundidade da análise estatística: enquanto o trabalho de Iaquinta e Fouilloux [4] apresenta resultados agregados, este experimento separou os dados em 23 grupos distintos de tamanhos de payload (1 B a 4 MB) e realizou uma análise aprofundada das repetições dos testes com $N = 30$ amostras por grupo. Essa granularidade permitiu identificar comportamentos sutis, como a anomalia observada em 256 B e a transição de overhead nos micro-payloads, que não foram explorados no trabalho anterior.

Apesar dessas diferenças metodológicas, os resultados são consistentes entre os estudos: ambos confirmam que o overhead do contêiner é desprezível para payloads grandes ($\geq$ 32 KB) e que a equivalência estatística é alcançada em largura de banda de pico. No entanto, nossa análise detalhada das variações entre repetições revela padrões de estabilidade que reforçam a confiabilidade dos benchmarks em contêineres para uso em ambientes HPC produtivos.

8 Conclusão

O experimento demonstrou que contêineres Apptainer/Singularity, quando configurados corretamente com mapeamento de dispositivos InfiniBand e bibliotecas UCX compatíveis, oferecem desempenho equivalente à execução nativa em clusters HPC. A largura de banda atingiu $\approx$ 96%–97% do limite teórico da rede InfiniBand HDR100, confirmando a viabilidade de contêineres para aplicações MPI de alta performance sem perda significativa de throughput.

9 Referências

Referências

[1] Apptainer. Apptainer/singularity documentation, 2026. Disponível em: `https://apptainer.org/`. Acesso em: 8 out. 2026.

[2] M. Champ et al. Mpich: uma implementação de código aberto de mpi, 2026. Disponível em: `https://www.mpich.org/`. Acesso em: 8 out. 2026.

[3] G. Dream et al. Spack package manager, 2026. Disponível em: `https://spack.io/`. Acesso em: 8 out. 2026.

[4] Jean Iaquinta and Anne Fouilloux. Unlocking the Potential of Containers in Scientific Computing to Achieve Bitwise Reproducibility, Portability and Performance. In Abdulrahman Azab and Tomasz Malkiewicz, editors, *Nordic E-Infrastructure Tomorrow*, pages 45–60. Springer Nature Switzerland. `doi:10.1007/978-3-031-86240-3_4`.

[5] Intel Corporation. Ucx – unified communication x, 2026. Disponível em: `https://github.com/openucx/ucx`. Acesso em: 8 out. 2026.

[6] G. Moore et al. Osu micro-benchmarks, 2026. Disponível em: `https://mvapich.cse.ohio-state.edu/benchmarks/`. Acesso em: 8 out. 2026.

[7] NVIDIA. Nvidia hpc container maker (hpccm), 2026. Disponível em: `https://github.com/NVIDIA/hpc-container-maker`. Acesso em: 8 out. 2026.

[8] SciPy Developers. scipy.stats.mannwhitneyu — mann-whitney u test, 2026. Disponível em: `https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.mannwhitneyu.html`. Acesso em: 8 out. 2026.

[9] Statsmodels Developers. statsmodels.stats.multitest — múltiplos testes, 2026. Disponível em: `https://www.statsmodels.org/stable/generated/statsmodels.stats.multitest.multipletests.html`. Acesso em: 8 out. 2026.